Assignment 2

Theory: Arithmetic of CNNs

For each spatial dimension, convolution or pooling (with dilation 1 and floor rounding) uses

```
output = floor((input + 2 x padding - kernel) / stride) + 1.
```

Convolution output depth equals the number of filters. Pooling preserves depth.

Question 1

Input: 32 × 32 × 3; 8 filters; kernel 5 × 5; stride 1; padding 0.

```
output height = output width = floor((32 + 0 - 5) / 1) + 1 = 28.
```

Output: 28 × 28 × 8. Each filter spans all three input channels and produces one output feature map.

Question 2

With “same” padding and stride 1, spatial dimensions stay unchanged. For an odd kernel of size 5, padding is (5 – 1) / 2 = 2 pixels on each side.

```
output height = output width = floor((32 + 2 x 2 - 5) / 1) + 1 = 32.
```

Output: 32 × 32 × 8, compared with 28 × 28 × 8 without padding.

Question 3

Input: 64 × 64; kernel 3 × 3; stride 2; padding 0.

```
output height = output width = floor((64 - 3) / 2) + 1
= floor(30.5) + 1 = 31.
```

Output spatial size: 31 × 31. A partial window at the boundary is not included. Output depth would equal the number of filters, which the question does not specify.

Question 4

Input feature map: 16 × 16; pooling kernel 2 × 2; stride 2; padding 0.

```
output height = output width = floor((16 - 2) / 2) + 1 = 8.
```

Output: 8 × 8 for each feature map. If there are C input channels, the complete output shape is 8 × 8 × C.

Question 5

Each convolution uses kernel 3 × 3, stride 1, and “same” padding, so padding is 1 on each side.

```
first layer: floor((128 + 2 x 1 - 3) / 1) + 1 = 128
second layer: floor((128 + 2 x 1 - 3) / 1) + 1 = 128.
```

Output spatial shape: 128 × 128. The final channel count equals the second convolution’s number of filters; that number is not given.

Question 6

`model.train()` puts a PyTorch module and its child modules into training mode. It changes the behavior of layers such as Dropout and BatchNorm. It does not enable gradients by itself; gradient recording is controlled separately.

A newly constructed model starts in training mode. If the model has just been created and no earlier code called `model.eval()`, removing the initial `model.train()` usually makes no difference. The assignment’s CNN contains convolution, ReLU, max-pooling, flattening, and linear layers, whose behavior does not change between training and evaluation modes. Consequently, for this exact architecture, removing that line alone does not change its predictions or stop backpropagation.

If a model with Dropout or BatchNorm was previously switched to evaluation mode, omitting `model.train()` leaves those layers in evaluation mode during subsequent training. Dropout would remain disabled, and BatchNorm would use stored running statistics rather than updating them from training batches. Keeping `model.train()` explicitly before the training loop is therefore good practice, especially after validation.